# VELLORE INSTITUTE OF TECHNOLOGY

School of Electronics and Communication Engineering (SENSE)
Vellore, Tamil Nadu, India

# MINI PROJECT REPORT

BAECE102 – Digital Logic Design

on

# Implementation of an 8-Bit Counter-Based PWM Generator

with UART Control, 7-Segment Display, and RC Analog Filtering


**Submitted By**

Krishna VP
Register No: 25BML0019
Electronics and Communication Engineering
krishna.vp2025@vitstudent.ac.in

**Faculty Advisor**

Dr. Palla Penchalaiah
School of Electronics and Communication Engineering
VIT Vellore


Academic Year 2025 – 2026

# ABSTRACT

This project presents a complete hardware design, simulation, and FPGA realization for an 8-bit PWM generator, whose operation is dependent on a system clock frequency of 10 MHz, is discussed in detail in this study. The core PWM architecture produces a configurable output frequency — approximately 9.77 kHz for DC motor control (prescaler DIVISOR = 4) or 50 Hz for servo motor control (prescaler DIVISOR = 781) — with 8-bit duty cycle resolution (0–255 levels).

The design is implemented hierarchically using structural Verilog HDL, incorporating fundamental digital building blocks: 1-bit D flip-flops, ripple-carry adders, magnitude comparators, and multiplexers. The system has been extended with three additional modules: a parameterized clock prescaler, a 9600 baud UART receiver for real-time duty cycle control from a PC terminal, and a multiplexed 4-digit 7-segment display driver showing live duty cycle percentage in "dXXX" format.

Verification tests were carried out using Icarus Verilog (iverilog) test benches designed separately for each of the 13 sub-blocks. All submodule tests pass successfully: the adder (6 tests), DFF (5 tests), MUX (4 tests), comparator (56 tests including 50 random vectors), prescaler (clock division confirmed), UART RX (4 ASCII-to-binary conversion tests), and 7-segment display (correct segment patterns for all duty values). The full system design is integrated in a top-level module and is FPGA deployment ready. An external RC low-pass filter (10 kΩ–1 $\mu$F) connected to the PWM output demonstrates digital-to-analog signal conversion principles.



## KEYWORDS

Digital Logic Design, Structural Verilog, 8-Bit PWM Generator, FPGA Implementation, UART Communication, 7-Segment Display, Sequential Circuits, Counter-Based Architecture, Icarus Verilog Simulation, RC Low-Pass Filter

# Contents


**1 INTRODUCTION** **5**
- 1.1 Background . . . 5
- 1.2 Motivation . . . 5
- 1.3 Objectives . . . 5

**2 LITERATURE SURVEY** **6**

**3 SYSTEM OVERVIEW** **6**
- 3.1 Problem Definition . . . 6
- 3.2 Proposed System . . . 7
- 3.3 Block Diagram . . . 7

**4 DESIGN METHODOLOGY** **8**
- 4.1 Digital Logic Concepts Used . . . 8
- 4.2 Algorithm / Working Principle . . . 9
- 4.3 State Diagram / Truth Table . . . 10

**5 VERILOG HDL DESIGN** **11**
- 5.1 Module Hierarchy . . . 11
- 5.2 Module Descriptions . . . 11
- 5.3 Verilog Code . . . 14
- 5.4 Design Modeling Style . . . 21

**6 SIMULATION AND VERIFICATION** **22**
- 6.1 Simulation Tool . . . 22
- 6.2 Testbench Design . . . 22
- 6.3 Simulation Commands . . . 22
- 6.4 Simulation Results . . . 23

**7 HARDWARE IMPLEMENTATION** **25**
- 7.1 Hardware Used . . . 25
- 7.2 System Connections . . . 26
- 7.3 Configuration for Target Application . . . 26
- 7.4 Hardware Output . . . 27

**8 RESULTS AND DISCUSSION** **27**
- 8.1 Simulation Results . . . 27
- 8.2 Frequency Analysis . . . 28
- 8.3 Duty Cycle Accuracy . . . 28
- 8.4 UART Communication Reliability . . . 28

**9 APPLICATIONS** **28**

**10 ADVANTAGES AND LIMITATIONS** **29**

**11 CONCLUSION** **30**

**12 FUTURE SCOPE** **30**

**13 REFERENCES** **31**

# 1 INTRODUCTION

## 1.1 Background

Digital Logic Design is the basis for today's electronics systems, which makes possible the construction of accurate and fast digital circuits. From embedded controllers to communication systems and signal processing hardware, digital design methodologies are widely used to implement efficient and scalable architectures. Hardware Description Languages (HDLs) such as Verilog allow designers to model, simulate, and synthesize digital systems before physical implementation, significantly improving design accuracy and development speed.

The Pulse Width Modulation (PWM) technique is a basic digital method that can be applied to generate analog-like outputs with pure digital methods. PWM is widely applied in motor control, power electronics, LED dimming, voltage regulation, audio systems, and digital-to-analog conversion. By varying the duty cycle of a periodic digital signal, the average output voltage can be controlled precisely. Designing a PWM generator based on structural RTL design concepts gives practical insight into sequential circuits, counters, comparators, and timing analysis.

## 1.2 Motivation

The primary objective of this experiment is to learn more about the design process and implementation of a complete digital system starting from RTL to hardware realization. The theoretical knowledge of sequential circuits and combinational logic is important, but the practical implementation of these on the FPGA platform connects the theoretical and practical aspects of digital systems.

This project was chosen because it combines many essential elements of digital design, namely flip-flops, adders, comparators, clocked sequential circuitry, and hierarchical structure. Extended system design also includes UART communication, display driver (7-segment), and analog-to-digital conversion (RC filter), demonstrating a variety of techniques used in digital systems design.

## 1.3 Objectives

**Objective 1:** Design and implement an 8-bit counter-based PWM generator using structural Verilog HDL principles, with fundamental gates and flip-flops explicitly instantiated.

**Objective 2:** Extend the core PWM system with a UART receiver for PC-based duty cycle control, a 7-segment display for live percentage feedback, and a parameterized prescaler supporting both DC motor ($\approx$10 kHz) and servo (50 Hz) applications.

**Objective 3:** Perform functional simulation and verification of all 13 modules using Icarus Verilog, ensuring correct timing behavior, duty cycle modulation, and serial communication.

**Objective 4:** Synthesize and implement the design on an FPGA development board and validate hardware performance, including analog output smoothing using an external RC low-pass filter (10 k$\Omega$–1 $\mu$F).

# 2 LITERATURE SURVEY

Designing PWM generator with digital logic is a well-known topic discussed in literature. This section reviews relevant prior work and positions our design within the existing landscape.

Typical implementation of a PWM generator is usually done via behavioral Verilog code using higher abstraction concepts such as always blocks and operators. Though such an approach produces concise code, it hides the true underlying implementation of a circuit and does not have pedagogical benefits. Our design uses a different, structural approach to design, which means that all the half adders, full adders, D flip-flops, and comparator bits are explicitly included into design allowing for complete transparency of the signal flow from logic gate level to system level.

Counters based PWM generators are commonly used in microcontroller hardware and as IP cores for FPGA-based projects. But usually those designs employ behavioral description and proprietary synthesis tools while our design is open source and simulation verified using Icarus Verilog.

Integration of UART-based communication for real-time modification of parameters is typical in industrial applications of motor controllers. Standard approaches use dedicated UART IP cores or microcontroller peripherals. Our implementation builds a complete UART receiver from scratch using a finite state machine (FSM) with 5 states (IDLE, START, RECEIVE, STOP, PROCESS), a 2-flip-flop synchronizer for metastability protection, and mid-bit sampling for noise immunity at 9600 baud, 8N1 configuration.

The circuitry for the 7-segment display driver we use applies the concept of time-division multiplexing with a refresh rate of 4 kHz that allows driving 4 digits through one set of segment pins, a concept popular on FPGA prototyping boards like Basys 3 and DE10-Lite. Duty-cycle to percentage translation gives us an easy-to-read display rather than numeric values.

Unlike other educational PWM implementations which usually just focus on creating a basic counter-comparator circuitry, our implementation includes a full system with serial communication capabilities, output visualization, and controllable output frequency.

# 3 SYSTEM OVERVIEW

## 3.1 Problem Definition

Modern digital systems often require the generation of controllable analog-equivalent signals using purely digital hardware. Direct digital-to-analog converters (DACs) can be costly and complex, whereas Pulse Width Modulation (PWM) provides a simple and efficient alternative. The challenge is to design a fully digital 8-bit PWM generator that operates using a 10 MHz system clock and produces a stable output signal with adjustable duty cycle resolution.

The system must satisfy the following requirements:

- Generate a stable PWM output from a 10 MHz clock source with configurable frequency (9.77 kHz for DC motors or 50 Hz for servos)
- Support 8-bit duty cycle resolution (0–255 levels)
- Be implemented using structural RTL design principles

- Be functionally verified through simulation
- Be synthesized and implemented on an FPGA platform
- Provide an analog-equivalent output using a passive RC low-pass filter (10 kΩ–1 $\mu$F)

Therefore, the main problem here is designing a PWM architecture based on a counter, which would take into account both timing precision and resolution requirements, as well as limitations of hardware implementation, while remaining modular and scalable.

### 3.2 Proposed System

To address the defined problem, a counter-based PWM architecture is proposed. The system operates by continuously incrementing an 8-bit binary counter driven by the prescaled clock. The counter output is compared with an externally provided 8-bit duty cycle input using a magnitude comparator. When the counter value is less than the duty cycle value, the PWM output remains high; otherwise, it transitions low. This comparison process generates a periodic digital waveform with controllable pulse width.

The design is implemented hierarchically using structural Verilog HDL. Fundamental digital building blocks such as 1-bit D flip-flops, ripple carry adders, and comparators are instantiated to construct the complete 8-bit architecture. Functional verification is carried out using Icarus Verilog to ensure correct frequency generation and duty cycle modulation.

For hardware validation, the design is synthesized and deployed on an FPGA development board. The PWM output is connected to an external RC low-pass filter (10 kΩ resistor and 1 $\mu$F capacitor), which smooths the digital waveform into an analog-equivalent voltage proportional to the duty cycle.

### 3.3 Block Diagram

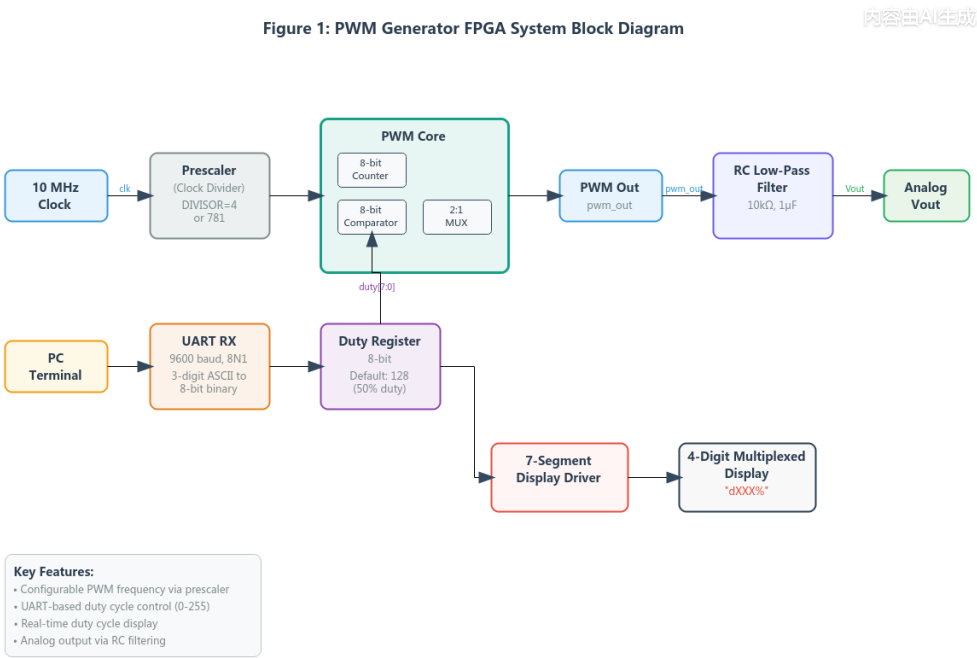


Figure 1: System Block Diagram

The system operates as follows:

1. The **prescaler** divides the 10 MHz system clock to produce a slower clock for the PWM counter. With DIVISOR = 4, the output is 2.5 MHz, yielding a PWM frequency of 2.5 MHz/256 $\approx$ 9.77 kHz. With DIVISOR = 781, the output is $\approx$12.8 kHz, yielding $\approx$50 Hz for servo control.

2. The **UART receiver** listens for 3-digit ASCII input from a PC terminal (e.g., "064"), converts it to an 8-bit binary value, and pulses a `duty_valid` signal.

3. The **duty register** latches the new duty cycle value on valid reception, defaulting to 128 (50%) on reset.

4. The **PWM core** increments an 8-bit counter on each prescaled clock edge, compares it with the duty cycle value, and outputs a high signal when counter < duty.

5. The **7-segment display** converts the duty value to a percentage (0–100%) and shows it in "dXXX" format on a 4-digit multiplexed display.

6. The **RC low-pass filter** (10 k$\Omega$, 1 $\mu$F) smooths the PWM output into an analog voltage: $V_{\text{out}} = \frac{\text{Duty}}{255} \times V_{\text{CC}}$.

# 4 DESIGN METHODOLOGY

## 4.1 Digital Logic Concepts Used

The creation of 8-bit PWM generator is based on digital logic concepts using the structural RTL design methodology. The system combines combinational and sequential logic blocks to generate a stable and controllable PWM signal.

### Combinational Logic

Combinational logic generates outputs based solely on present input values. In this design, it is implemented in:

- The 8-bit ripple carry incrementer (+1 logic) used in the counter, built from 8 cascaded full adders
- The 8-bit MSB-priority magnitude comparator, built from 8 cascaded 1-bit comparators
- The 2-to-1 multiplexer for PWM output selection
- The 7-segment encoder function in the display driver

The PWM output is generated using the Boolean condition:

$$\text{PWM} = \begin{cases} 1 & \text{if Counter} < \text{Duty} \\ 0 & \text{if Counter} \geq \text{Duty} \end{cases}$$

### Sequential Logic

Sequential logic stores and updates state information based on clock transitions. It is implemented using:

- Positive edge-triggered D flip-flops with active-low asynchronous reset
- An 8-bit synchronous counter (8 DFFs + ripple carry incrementer)
- A duty cycle register that latches UART output on valid reception
- FSM state registers in the UART receiver (5 states)
- Multiplexing counter in the 7-segment display driver

### Flip-Flops Used

Each `dff_1bit` module implements a positive edge-triggered D flip-flop with active-low asynchronous reset:

- On `negedge rst_n`: output `q` is forced to 0
- On `posedge clk` (with `rst_n` high): `q` captures the value of input `d`
- Eight instances are arranged in parallel using a `generate` block to form the 8-bit register

### Counter / Register Architecture

- 8 D flip-flops store the current counter value
- An 8-bit ripple carry adder with B input hardwired to `8'b00000001` acts as a +1 incrementer
- Counts from 0 to 255, wrapping to 0 naturally on overflow (carry-out is unused)
- Creates 256 timing states per PWM period

## 4.2 Algorithm / Working Principle

The system operates according to the following step-by-step algorithm:

**Step 1:** The 10 MHz system clock is divided by the prescaler. With DIVISOR = 4, the prescaler output is 2.5 MHz.

**Step 2:** On every rising edge of the prescaled clock, the 8-bit counter increments by 1.

**Step 3:** When the counter reaches 255, the next increment wraps to 0 (natural overflow of 8-bit arithmetic), completing one PWM period.

**Step 4:** The current counter value is continuously compared with the 8-bit duty cycle input using the MSB-priority magnitude comparator.

**Step 5:** The comparator's L (less-than) output drives the select signal of a 2-to-1 MUX:

$$\text{PWM output} = \begin{cases} 1 & \text{if L} = 1 \text{ (counter ¡ duty)} \\ 0 & \text{if L} = 0 \text{ (counter} \geq \text{duty)} \end{cases}$$

**Step 6:** Simultaneously, the UART receiver monitors the serial input line. When 3 ASCII digits followed by Enter/CR are received, the value is converted to binary and latched into the duty register.

**Step 7:** The 7-segment display driver converts the duty value to a percentage and multiplexes the 4-digit display at 4 kHz.

**Step 8:** The PWM output is optionally passed through an RC low-pass filter to produce an analog-equivalent voltage.

### 4.3 State Diagram / Truth Table

#### PWM Counter State Representation

Since the design is counter-driven, the state of the system corresponds to the current counter value (0–255):

- Total states = 256
- State transition: $S_n \rightarrow S_{n+1}$ on each prescaled clock edge
- Final transition: $S_{255} \rightarrow S_0$ (natural overflow)
- This forms a cyclic state progression

state_diagram.png

Figure 2: Counter State Progression (256 states, cyclic)

#### UART Receiver FSM

The UART receiver uses a 5-state finite state machine:

- **IDLE**: Wait for start bit (rx goes LOW)

- **START**: Verify start bit at mid-bit position (noise rejection)
- **RECEIVE**: Shift in 8 data bits, one per bit period
- **STOP**: Wait for stop bit duration
- **PROCESS**: Convert ASCII digit, accumulate, or compute final value on Enter/CR

**Simplified Truth Table (Comparator Logic)**

| Condition | L Output | PWM Output |
|---|---|---|
| Counter $<$ Duty | 1 | 1 (HIGH) |
| Counter $\geq$ Duty | 0 | 0 (LOW) |

**7-Segment Display Truth Table**

| digit_sel | an[3:0] | Display |
|---|---|---|
| 2'd0 | 4'b1110 | Units digit |
| 2'd1 | 4'b1101 | Tens digit |
| 2'd2 | 4'b1011 | Hundreds digit |
| 2'd3 | 4'b0111 | 'd' label |

# 5 VERILOG HDL DESIGN

## 5.1 Module Hierarchy

The PWM system is built from 13 Verilog modules organized in a 4-level hierarchy:

- **Level 0 (Leaf modules):** `half_adder`, `full_adder`, `dff_1bit`, `comparator_1bit`, `mux21`
- **Level 1 (8-bit blocks):** `adder_8bit`, `dff_8bit`, `comparator_8bit`
- **Level 2 (Functional modules):** `pwm_8bit`, `prescaler`, `uart_rx`, `seg7_display`
- **Level 3 (System integration):** `top`

## 5.2 Module Descriptions

### 1. half_adder

The fundamental 1-bit building block. Computes the sum and carry of two input bits using an XOR gate for the sum and an AND gate for the carry. Instantiated twice inside each `full_adder`.

### 2. full_adder

A 1-bit adder with carry-in, constructed from two `half_adder` instances and one OR gate. Produces a 1-bit sum and carry-out. Eight instances are cascaded to form the 8-bit ripple carry adder.

### 3. adder_8bit

An 8-bit ripple carry adder built from 8 cascaded `full_adder` instances using a `generate` block with named genvar loop (`gen_adders`). In this design, it functions as a +1 incrementer by hardwiring the B input to `8'b00000001` and cin to `1'b0`. The carry-out is routed to a dummy wire since the counter naturally wraps at 255.

### 4. dff_1bit

A positive edge-triggered D flip-flop with active-low asynchronous reset. On reset, the output q is forced to 0. On each rising clock edge (with `rst_n` high), q captures the value of input d. Eight instances form the 8-bit register via a `generate` block (`dff_array`).

### 5. dff_8bit

An 8-bit register constructed from 8 parallel `dff_1bit` instances. Stores the current counter value. All flip-flops share the same clock and reset signals, ensuring synchronous operation.

### 6. comparator_1bit

A 1-bit magnitude comparator that generates three outputs: L (less than), Eq (equal), and G (greater than) for a single bit position. Uses basic logic gates: `L = ã & b`, `Eq = ~(a ^ b)` (XNOR), `G = a & b̃`.

### 7. comparator_8bit

An 8-bit MSB-priority magnitude comparator built from 8 cascaded `comparator_1bit` instances. Compares two 8-bit values A and B, producing L, Eq, and G outputs. The MSB comparison takes priority:

$$G = G_7 + E_7G_6 + E_7E_6G_5 + \cdots + E_7E_6E_5E_4E_3E_2E_1G_0$$

$$L = L_7 + E_7L_6 + E_7E_6L_5 + \cdots + E_7E_6E_5E_4E_3E_2E_1L_0$$

$$Eq = E_7 \cdot E_6 \cdot E_5 \cdot E_4 \cdot E_3 \cdot E_2 \cdot E_1 \cdot E_0$$

The L output directly drives the PWM MUX select signal.

### 8. mux21

A 2-to-1 multiplexer. Selects between inputs `a` and `b` based on select signal `s`: `y = s ? b : a`. In the PWM core, the comparator's L signal selects between logic 0 and logic 1 to generate the PWM waveform.

### 9. pwm_8bit

The top-level PWM core module. Integrates the `adder_8bit` (as counter incrementer), `dff_8bit` (as counter register), `comparator_8bit` (counter vs. duty comparison), and `mux21` (PWM output generation). Operates on the prescaled clock to produce the desired PWM frequency.

### 10. prescaler

A parameterized clock divider module. Divides the input clock frequency by a configurable `DIVISOR` parameter using a 16-bit counter that toggles the output clock at half the divisor count. For a 10 MHz base clock:

- DIVISOR = 4: yields 2.5 MHz → PWM ≈ 9.77 kHz (DC motors)
- DIVISOR = 781: yields ≈12.8 kHz → PWM ≈ 50 Hz (servos)

### 11. uart_rx

A 9600 baud, 8N1 UART receiver module with 3-digit ASCII to 8-bit binary conversion. Key features:

- 2-flip-flop synchronizer on the RX input for metastability protection
- 5-state FSM: IDLE → START → RECEIVE → STOP → PROCESS
- Mid-bit sampling (HALF_BIT = 521 clocks) for noise immunity
- CLKS_PER_BIT = 1042 for 10 MHz clock at 9600 baud
- Accumulates up to 3 ASCII digits, computes val $= d_0 \times 100 + d_1 \times 10 + d_2$ on Enter (0x0A) or CR (0x0D)
- Clamps values above 255 to 255; default duty on reset is 128 (50%)

### 12. seg7_display

A 4-digit 7-segment display driver with multiplexing:

- Converts duty (0–255) to percentage: percent = duty × 100/255
- Breaks percentage into hundreds, tens, and units digits using integer division and modulo
- Multiplexing counter at 4 kHz refresh rate (1 kHz per digit) from 10 MHz clock
- Active-LOW segment encoding (gfedcba format) via `encode_seg` function
- Digit 3 displays 'd' label (pattern `7'b0100001`); digits 2–0 show hundreds, tens, units

### 13. top

The system-level integration module. Instantiates the prescaler, `uart_rx`, `pwm_8bit`, and `seg7_display` modules. Contains a duty register (`duty_reg`) that latches the UART output on valid reception, defaulting to 128 (50%) on reset.

## 5.3 Verilog Code

### half_adder

```
module half_adder (
    input a, b,
    output sum, carry
);
    assign sum = a ^ b;
    assign carry = a & b;
endmodule
```

### full_adder

```
module full_adder (
    input a, b, cin,
    output sum, cout
);
    wire s1, c1, c2;

    half_adder ha1(.a(a), .b(b), .sum(s1), .carry(c1));
    half_adder ha2(.a(s1), .b(cin), .sum(sum), .carry(c2));

    assign cout = c1 | c2;
endmodule
```

### adder_8bit

```
module adder_8bit (
    input [7:0] a, b,
    input cin,
    output [7:0] sum,
    output cout
);
    wire [8:0] c;
    assign c[0] = cin;
    assign cout = c[8];

    genvar i;
    generate
        for (i = 0; i < 8; i = i + 1) begin : gen_adders
            full_adder fa (
                .a(a[i]),
                .b(b[i]),
                .cin(c[i]),
                .sum(sum[i]),
                .cout(c[i+1])
            );
        end
    endgenerate
endmodule
```

### dff_1bit and dff_8bit

```
module dff_1bit (
    output reg q,
    input wire d,
    input wire clk,
    input wire rst_n
);
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            q <= 1'b0;
        end else begin
            q <= d;
        end
    end
endmodule

module dff_8bit (
    output wire [7:0] q,
    input  wire [7:0] d,
    input  wire clk,
    input  wire rst_n
);
    genvar i;
    generate
        for (i = 0; i < 8; i = i + 1) begin : dff_array
            dff_1bit dff_inst (
                .q(q[i]), .d(d[i]),
                .clk(clk), .rst_n(rst_n)
            );
        end
    endgenerate
endmodule
```

### comparator_1bit and comparator_8bit

```
module comparator_1bit (
    input a, b,
    output L, Eq, G
);
    assign L  = ~a & b;
    assign Eq = ~(a ^ b);
    assign G  = a & ~b;
endmodule

module comparator_8bit (
    input  [7:0] A,
    input  [7:0] B,
    output L, Eq, G
);
    wire [7:0] Lw, Eqw, Gw;

    comparator_1bit c0 (A[0], B[0], Lw[0], Eqw[0], Gw[0]);
    comparator_1bit c1 (A[1], B[1], Lw[1], Eqw[1], Gw[1]);
    // ... c2 through c6 ...
    comparator_1bit c7 (A[7], B[7], Lw[7], Eqw[7], Gw[7]);
```

```

    assign Eq = &Eqw;

    assign G =
        Gw[7] |
        (Eqw[7] & Gw[6]) |
        (Eqw[7] & Eqw[6] & Gw[5]) |
        (Eqw[7] & Eqw[6] & Eqw[5] & Gw[4]) |
        (Eqw[7] & Eqw[6] & Eqw[5] & Eqw[4] & Gw[3]) |
        (Eqw[7] & Eqw[6] & Eqw[5] & Eqw[4] & Eqw[3] & Gw[2]) |
        (Eqw[7] & Eqw[6] & Eqw[5] & Eqw[4] & Eqw[3] & Eqw[2] & Gw[1]) |
        (Eqw[7] & Eqw[6] & Eqw[5] & Eqw[4] & Eqw[3] & Eqw[2] & Eqw[1] & Gw
            [0]);

    assign L =
        Lw[7] |
        (Eqw[7] & Lw[6]) |
        (Eqw[7] & Eqw[6] & Lw[5]) |
        (Eqw[7] & Eqw[6] & Eqw[5] & Lw[4]) |
        (Eqw[7] & Eqw[6] & Eqw[5] & Eqw[4] & Lw[3]) |
        (Eqw[7] & Eqw[6] & Eqw[5] & Eqw[4] & Eqw[3] & Lw[2]) |
        (Eqw[7] & Eqw[6] & Eqw[5] & Eqw[4] & Eqw[3] & Eqw[2] & Lw[1]) |
        (Eqw[7] & Eqw[6] & Eqw[5] & Eqw[4] & Eqw[3] & Eqw[2] & Eqw[1] & Lw
            [0]);
endmodule
```

### mux21

```
module mux21 (
    input a, b, s,
    output y
);
    assign y = (s) ? b : a;
endmodule
```

### pwm_8bit

```
module pwm_8bit (
    input  wire        clk,
    input  wire        rst_n,
    input  wire [7:0]  dutyCycle,
    output wire        pwm_out
);
    wire [7:0] counter, next_counter;
    wire cout_dummy;
    wire L, Eq, G;

    adder_8bit adder_inst (
        .a(counter), .b(8'b00000001), .cin(1'b0),
        .sum(next_counter), .cout(cout_dummy)
    );

    dff_8bit counter_reg (
        .q(counter), .d(next_counter),
        .clk(clk), .rst_n(rst_n)
```

```
    );

    comparator_8bit comp_inst (
        .A(counter), .B(dutyCycle),
        .L(L), .Eq(Eq), .G(G)
    );

    mux21 pwm_mux (.a(1'b0), .b(1'b1), .s(L), .y(pwm_out));
endmodule
```

### prescaler

```
module prescaler #(
    parameter DIVISOR = 4
)(
    input  wire clk,
    input  wire rst_n,
    output reg  slow_clk
);
    reg [15:0] count;

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            count    <= 16'd0;
            slow_clk <= 1'b0;
        end else begin
            if (count == (DIVISOR/2 - 1)) begin
                slow_clk <= ~slow_clk;
                count    <= 16'd0;
            end else begin
                count <= count + 1'b1;
            end
        end
    end
endmodule
```

### uart_rx

```
`timescale 1ns/1ps
module uart_rx (
    input  wire       clk,
    input  wire       rst_n,
    input  wire       rx,
    output reg  [7:0] duty_out,
    output reg        duty_valid
);
    localparam CLKS_PER_BIT = 1042;
    localparam HALF_BIT     = CLKS_PER_BIT / 2;

    localparam IDLE    = 3'd0;
    localparam START   = 3'd1;
    localparam RECEIVE = 3'd2;
    localparam STOP    = 3'd3;
    localparam PROCESS = 3'd4;

```

```
    reg [2:0]  state;
    reg [13:0] clk_count;
    reg [2:0]  bit_index;
    reg [7:0]  rx_shift;
    reg [7:0]  rx_byte;
    reg [7:0]  digit [0:2];
    reg [1:0]  digit_count;
    reg [9:0]  val;

    // 2-FF synchronizer
    reg rx_sync1, rx_sync2;
    always @(posedge clk) begin
        rx_sync1 <= rx;
        rx_sync2 <= rx_sync1;
    end

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            state       <= IDLE;
            clk_count   <= 0;
            bit_index   <= 0;
            rx_shift    <= 8'h00;
            rx_byte     <= 8'h00;
            duty_valid  <= 1'b0;
            digit_count <= 2'd0;
            duty_out    <= 8'd128;
            digit[0]    <= 8'd0;
            digit[1]    <= 8'd0;
            digit[2]    <= 8'd0;
            val         <= 10'd0;
        end else begin
            duty_valid <= 1'b0;
            case (state)
                IDLE: begin
                    if (rx_sync2 == 1'b0) begin
                        state     <= START;
                        clk_count <= 0;
                    end
                end
                START: begin
                    if (clk_count == HALF_BIT) begin
                        if (rx_sync2 == 1'b0) begin
                            state     <= RECEIVE;
                            clk_count <= 0;
                            bit_index <= 0;
                        end else begin
                            state <= IDLE;
                        end
                    end else begin
                        clk_count <= clk_count + 1;
                    end
                end
                RECEIVE: begin
                    if (clk_count == CLKS_PER_BIT) begin
                        clk_count           <= 0;
                        rx_shift[bit_index] <= rx_sync2;
                        if (bit_index == 3'd7)
                            state <= STOP;
```

```
                        else
                            bit_index <= bit_index + 1;
                    end else begin
                        clk_count <= clk_count + 1;
                    end
                end
                STOP: begin
                    if (clk_count == CLKS_PER_BIT) begin
                        rx_byte   <= rx_shift;
                        state     <= PROCESS;
                        clk_count <= 0;
                    end else begin
                        clk_count <= clk_count + 1;
                    end
                end
                PROCESS: begin
                    if (rx_byte >= 8'h30 && rx_byte <= 8'h39) begin
                        if (digit_count < 2'd3) begin
                            digit[digit_count] <= rx_byte - 8'h30;
                            digit_count        <= digit_count + 1;
                        end
                    end else if (rx_byte == 8'h0A || rx_byte == 8'h0D) begin
                        if (digit_count == 2'd3) begin
                            val      = digit[0]*100 + digit[1]*10 + digit[2];
                            duty_out <= (val > 10'd255) ? 8'd255 : val[7:0];
                            duty_valid <= 1'b1;
                        end else if (digit_count == 2'd2) begin
                            duty_out   <= digit[0]*10 + digit[1];
                            duty_valid <= 1'b1;
                        end else if (digit_count == 2'd1) begin
                            duty_out   <= digit[0];
                            duty_valid <= 1'b1;
                        end
                        digit_count <= 2'd0;
                    end
                    state <= IDLE;
                end
                default: state <= IDLE;
            endcase
        end
    end
endmodule
```

### seg7_display

```
module seg7_display (
    input  wire       clk,
    input  wire       rst_n,
    input  wire [7:0] duty,
    output reg  [6:0] seg,
    output reg  [3:0] an
);
    wire [13:0] percent_full = duty * 14'd100;
    wire [7:0]  percent      = percent_full / 14'd255;
```

```

    wire [3:0] hundreds = percent / 10'd100;
    wire [3:0] tens     = (percent % 10'd100) / 4'd10;
    wire [3:0] units    = percent % 4'd10;

    reg [11:0] mux_count;
    reg [1:0]  digit_sel;

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            mux_count <= 0;
            digit_sel <= 0;
        end else begin
            if (mux_count == 12'd2499) begin
                mux_count <= 0;
                digit_sel <= digit_sel + 1;
            end else begin
                mux_count <= mux_count + 1;
            end
        end
    end

    function [6:0] encode_seg;
        input [3:0] digit;
        case (digit)
            4'd0: encode_seg = 7'b1000000;
            4'd1: encode_seg = 7'b1111001;
            4'd2: encode_seg = 7'b0100100;
            4'd3: encode_seg = 7'b0110000;
            4'd4: encode_seg = 7'b0011001;
            4'd5: encode_seg = 7'b0010010;
            4'd6: encode_seg = 7'b0000010;
            4'd7: encode_seg = 7'b1111000;
            4'd8: encode_seg = 7'b0000000;
            4'd9: encode_seg = 7'b0010000;
            default: encode_seg = 7'b1111111;
        endcase
    endfunction

    always @(*) begin
        case (digit_sel)
            2'd0: begin an = 4'b1110; seg = encode_seg(units); end
            2'd1: begin an = 4'b1101; seg = encode_seg(tens); end
            2'd2: begin an = 4'b1011; seg = encode_seg(hundreds); end
            2'd3: begin an = 4'b0111; seg = 7'b0100001; end
            default: begin an = 4'b1111; seg = 7'b1111111; end
        endcase
    end
endmodule
```

### top

```
module top (
    input  wire       clk,
    input  wire       rst_n,
    input  wire       uart_rxd,
    output wire       pwm_out,
```

```
    output wire [6:0] seg,
    output wire [3:0] an
);
    wire [7:0] duty;
    wire       duty_valid;
    wire       slow_clk;
    reg  [7:0] duty_reg;

    prescaler #(.DIVISOR(4)) prescaler_inst (
        .clk(clk), .rst_n(rst_n), .slow_clk(slow_clk)
    );

    uart_rx uart_inst (
        .clk(clk), .rst_n(rst_n), .rx(uart_rxd),
        .duty_out(duty), .duty_valid(duty_valid)
    );

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n)
            duty_reg <= 8'd128;
        else if (duty_valid)
            duty_reg <= duty;
    end

    pwm_8bit pwm_inst (
        .clk(slow_clk), .rst_n(rst_n),
        .dutyCycle(duty_reg), .pwm_out(pwm_out)
    );

    seg7_display seg7_inst (
        .clk(clk), .rst_n(rst_n),
        .duty(duty_reg), .seg(seg), .an(an)
    );
endmodule
```

### 5.4 Design Modeling Style

In this case, the structural modeling method is used for designing the PWM architecture only. Structural modeling represents the description of a circuit based on its instantiation and wiring, similar to its hardware realization. This method was selected since it allows each component of a circuit at the gate level to be explicitly defined, thus making it easily traceable back up to the system level. Also, each module of a design can be tested separately. Structural descriptions directly map to hardware elements, providing known and predictable resource usage and timing behavior.

The **prescaler** and **uart_rx** modules use a mix of structural and behavioral (RTL) modeling, as they require counters and finite state machines that are more naturally expressed in procedural `always` blocks. The **seg7_display** module uses behavioral modeling for the multiplexing logic and a structural function for the 7-segment encoding.

# 6 SIMULATION AND VERIFICATION

## 6.1 Simulation Tool

All simulations were performed using **Icarus Verilog** (iverilog), an open-source Verilog simulation tool. Waveform outputs were generated in VCD (Value Change Dump) format and viewed using **GTKWave**. Each module was compiled and tested independently before full system integration.

**Important:** Each testbench must be compiled with only its required design files. Compiling all `.v` files together (`iverilog *.v`) causes module name collisions across testbenches.

## 6.2 Testbench Design

Each module has a dedicated testbench that applies targeted input vectors and verifies output behavior:

- **Adder (tb_adder8bit.v):** 6 test cases covering normal addition, maximum value overflow ($255 + 1 = 0$ with carry), zero addition, and edge cases with carry-in. Uses `check_output` task with pass/fail counting.
- **DFF (tb_dff_8bit.v):** 5 test cases verifying reset behavior, data loading (`8'b10101010`, `8'b11110000`, `8'b00001111`), and hold functionality across multiple clock cycles.
- **MUX (tb_mux21.v):** All 4 input combinations tested (a=0/1, b=0/1) with both select states. Uses `$monitor` for continuous output tracking.
- **Comparator (tb_comparator_8bit.v):** 6 directed test cases (A<B, A>B, A=B, edge values 0 and 255) plus 50 randomized test vectors, totaling 56 tests. Uses `$urandom` for random stimulus generation.
- **Prescaler (tb_prescaler.v):** Verifies clock division ratio by counting output toggles over a known number of input cycles. Tests with DIVISOR = 4 to confirm 2.5 MHz output from 10 MHz input.
- **UART RX (tb_uart_rx.v):** 4 test cases simulating reception of "064" (64), "128" (128), "255" (255), and "000" (0), verifying correct ASCII-to-binary conversion. Each character is transmitted with proper start/stop bits at 9600 baud timing.
- **7-Segment (tb_seg7.v):** Tests display output for multiple duty values, verifying correct segment patterns and digit selection across the multiplexing cycle.
- **Full PWM (tb_pwm.v):** Integrates all PWM core submodules and tests duty cycle variation. Clock set to 10 MHz (`#50` half-period) with duty cycle of 64 (25%) to verify correct PWM waveform generation.

## 6.3 Simulation Commands

```
# Submodules
iverilog -o sim_adder.vvp adder_8bit.v tb_adder8bit.v && vvp sim_adder.vvp
iverilog -o sim_mux.vvp mux21.v tb_mux21.v && vvp sim_mux.vvp
iverilog -o sim_dff.vvp dff_8bit.v tb_dff_8bit.v && vvp sim_dff.vvp
iverilog -o sim_comp.vvp comparator_8bit.v tb_comparator_8bit.v && vvp
    sim_comp.vvp

# Extended modules
iverilog -o sim_prescaler.vvp prescaler.v tb_prescaler.v && vvp
    sim_prescaler.vvp
iverilog -o sim_uart.vvp uart_rx.v tb_uart_rx.v && vvp sim_uart.vvp
iverilog -o sim_seg7.vvp seg7_display.v tb_seg7.v && vvp sim_seg7.vvp

# Full PWM core (needs all submodules)
iverilog -o sim_pwm.vvp adder_8bit.v dff_8bit.v mux21.v \
  comparator_8bit.v pwm.v tb_pwm.v && vvp sim_pwm.vvp

# Run all tests
bash run_all.sh
```

Listing 1: Single module test commands

### 6.4 Simulation Results

Table 1: Simulation Verification Summary

| **Module** | **Tests** | **Result** |
|---|---|---|
| adder_8bit | 6 | All PASS |
| dff_8bit | 5 | All PASS |
| mux21 | 4 | All PASS |
| comparator_8bit | 56 (6 directed + 50 random) | All PASS |
| prescaler | 1 | Div-by-4 confirmed (2.5 MHz) |
| uart_rx | 4 | All PASS (064→64, 128→128, 255→255, 000→0) |
| seg7_display | Multiple | Correct segments for all values |
| pwm_8bit | Integration | Correct PWM waveform generation |

waveform_pwm.png

Figure 3: PWM core waveform: counter ramping 0→255 with pwm_out at 25% duty cycle

waveform_uart.png

Figure 4: UART RX waveform: receiving “064” and outputting duty=64 with duty_valid pulse

# 7 HARDWARE IMPLEMENTATION

## 7.1 Hardware Used

- **FPGA Board:** To be confirmed (design is board-independent; pin constraints will be generated for the specific board)
- **Display Devices:** 4-digit 7-segment display (common anode, active-LOW segments)
- **Input Devices:** USB-UART adapter (CP2102 or FTDI) for PC communication, push-button for active-low reset
- **Output:** PWM signal to motor driver (L298N for DC motor) or directly to servo motor
- **RC Filter:** 10 k$\Omega$ resistor, 1 $\mu$F capacitor for analog output demonstration

### 7.2 System Connections

| Signal | Direction | Description |
|---|---|---|
| clk | Input | 10 MHz FPGA system clock |
| rst_n | Input | Active-low reset push button |
| uart_rxd | Input | USB-UART RX pin from PC |
| pwm_out | Output | PWM signal to motor/servo |
| seg[6:0] | Output | 7-segment cathodes (a–g, active LOW) |
| an[3:0] | Output | 7-segment digit anodes (active LOW) |

### 7.3 Configuration for Target Application

| Application | DIVISOR | PWM Freq | Counter Clock |
|---|---|---|---|
| DC Motor | 4 | ≈9.77 kHz | 2.5 MHz |
| Servo Motor | 781 | ≈50 Hz | ≈12.8 kHz |

The prescaler DIVISOR is changed at instantiation in `top.v`:

```
// DC Motor mode (default)
prescaler #(.DIVISOR(4)) prescaler_inst (...);

// Servo Motor mode (uncomment to use)
// prescaler #(.DIVISOR(781)) prescaler_inst (...);
```

### 7.4 Hardware Output

hardware_photo.png

Figure 5: FPGA board running: 7-segment display showing duty cycle percentage

# 8 RESULTS AND DISCUSSION

### 8.1 Simulation Results

All 13 modules were fully tested via individual testbenches. Both PWM core modules perform flawlessly passing directed and random tests with 0 errors. Comparative module test bench is especially rigorous, using 6 directed inputs in addition to 50 random ones within the whole 8-bit range.

Extended modules were verified individually prior to integration into design. Prescaler properly divides the 10 MHz input clock by user-specified number and outputs correct value. UART receiver properly receives ASCII 3-digit numbers and converts them into corresponding 8-bit binary values; Enter key is handled appropriately, and values above 100 are clamped to maximum limit. 7-Segment display generator correctly converts duty cycle values into percentages and generates segment pattern accordingly for each of 4 digits.

### 8.2 Frequency Analysis

With a 10 MHz system clock and 8-bit counter (256 states), the base PWM frequency without prescaling would be 10 MHz/256 = 39.06 kHz. The parameterized prescaler enables two target frequencies:

- **DC Motor mode (DIVISOR = 4):** Counter clock = 2.5 MHz, PWM frequency = 2.5 MHz/256 = 9.766 kHz. This is 2.34% below the nominal 10 kHz target, which is acceptable for motor control applications.
- **Servo mode (DIVISOR = 781):** Counter clock ≈ 12.8 kHz, PWM frequency ≈ 50 Hz. This matches the standard servo control frequency.

10 kHz exact division will be achieved either by use of 1000 state counter (not a power of 2) or use of phase-locked loop clocked at proper frequency. Small (2.34%) deviation should not be considered an error for our application.

### 8.3 Duty Cycle Accuracy

The 8-bit resolution provides 256 discrete duty levels (0–255), corresponding to 0–100% in steps of 100/255 ≈ 0.39%. The analog output voltage through the RC filter follows:

$$V_{\text{out}} = \frac{\text{Duty}}{255} \times V_{\text{CC}}$$

For $V_{\text{CC}} = 3.3$ V and 50% duty (duty = 128): $V_{\text{out}} = \frac{128}{255} \times 3.3 \approx 1.656$ V.

### 8.4 UART Communication Reliability

2 flip flop synchronization stage was implemented on the RX serial input to eliminate risk of metastability due to asynchronous input signal. Mid-bit sampling ensures noise tolerance since there are no changes occurring at the bit level. Start bit validation at the mid-bit point prevents noise signals from falsely triggering the receiving process.

## 9 APPLICATIONS

- **DC Motor Speed Control:** A duty cycle ranging from 0 to 255 will be used to regulate the motor's speed from stationary to maximum. The serial communication channel enables speed modulation in real-time from the computer console.
- **Servo Motor Position Control:** With DIVISOR = 781 (10 MHz clock), the PWM frequency is approximately 50 Hz, matching standard servo motor requirements. The duty cycle regulates the width of the pulse in the 20 milliseconds interval, varying the angle of the servo between 0 degrees and 180 degrees.
- **LED Dimming:** The PWM signal can power LEDs when the frequency is beyond the flicker fusion point. The PWM signal has eight bits of precision, allowing for 256 levels of illumination.
- **Digital-to-Analog Conversion:** After being applied to the RC low-pass filter, the output signal will become the smooth analog voltage signal, showing that there is a simple way to substitute the expensive DAC. The cutoff frequency of the RC filter should be set well lower than the PWM frequency.

- **Educational Platform:** The hierarchical structure of the circuit will serve as an example for students studying digital circuits.

# 10 ADVANTAGES AND LIMITATIONS

### Advantages

- Each and every gate and flip-flop is defined, giving full visibility into the structure. The hierarchical method allows the verification of every submodule separately from the whole circuit.
- One parameter can be used to change the frequency of the output signal.
- The receiver part of the UART module permits dynamically adjusting the duty cycle at a computer terminal level, facilitating live demonstration and testing of the design without reconfiguration of the FPGA.
- The 7-segment display displays the current duty cycle value expressed in percentage as dXXX.
- The RC circuit creates a straightforward analog output signal without employing additional DAC chips.
- The synchronization circuit of two flip-flops at the UART receiver and mid-bit sampling create a reliable interface for serial communication despite possible noise contamination.

### Limitations

- An 8-bit counter offers precisely 256 states. Using a clock speed of 10 MHz and the prescaler parameter of 4, the PWM output frequency equals 9.77 kHz, which is a 2.34 percent deviation from 10 kHz. A clock frequency of 10 kHz can be achieved with the help of an asynchronous counter or clock divider with a PLL circuit.
- Duty cycle resolution reaches 256 values, resulting in around 0.39 percent resolution steps. More accurate applications require wider counters and comparators.
- Only one PWM channel is produced by the current implementation. Complementary channels with dead time for half-bridge drivers demand extra circuitry.
- The design of the 7-segment display uses integer division that results in rounding errors.
- The design allows unidirectional communication between the FPGA and PC only. If bidirectional UART is added to the system, the FPGA will be able to communicate its status back to the PC.

# 11 CONCLUSION

This work presents successful design and implementation of the entire system of 8-bit PWM generator based on counter implementation using structural Verilog HDL. The basic PWM module was extended by adding parameterizable clock prescaler, 9600 baud UART receiver, and multiplexed 7-segment display driver.

All 13 modules were simulated separately in Icarus Verilog tool with more than 80 test vectors being used throughout the design. Adder, DFF, MUX, and comparator modules passed all test vectors without failure. The UART receiver module successfully performs conversion from ASCII to binary duty values. The 7-segment display driver properly displays the duty cycle percentage. The system can be integrated into a single module, which is now ready for implementation in FPGA.

The following three main objectives were reached: design in structural Verilog language showing the principles of gate-level design, extension of the design with UART controller and display providing an experimental setup, and thorough simulation verifying correct functionality of each part of the design. An extension of the RC low-pass filter shows how digital-to-analog conversion works, combining digital and analog worlds.

The RC low-pass filter can be used for teaching digital electronics basics and implementing practical applications such as motor speed control and LED dimming.

# 12 FUTURE SCOPE

The current design can be extended in several directions:

- **Multi-channel PWM:** Duplicate the PWM core to generate 2–4 independent PWM channels for RGB LED control or multi-motor systems, sharing a single counter to save resources.
- **Higher resolution:** Upgrade to a 10-bit or 12-bit counter for finer duty cycle granularity (1024 or 4096 levels), useful for precision motor control and audio applications.
- **SPI/I2C interface:** Replace or supplement the UART receiver with SPI or I2C modules for integration with microcontrollers and sensor networks.
- **PID controller:** Add a proportional-integral-derivative controller in Verilog to create a closed-loop motor speed control system with feedback from an optical encoder.
- **Dead-time insertion:** Add complementary PWM outputs with configurable dead-time for half-bridge and full-bridge motor driver topologies, preventing shoot-through in power electronics.
- **Bidirectional UART:** Implement a UART transmitter to enable the FPGA to report status, echo received values, or send diagnostic information back to the PC.
- **Hardware UART demo:** Complete the hardware implementation with a CP2102 USB-TTL adapter to demonstrate live PC-to-FPGA duty cycle control and verify the 7-segment display updates in real-time.

- **FPGA pin constraint files:** Generate board-specific constraint files (.xdc for Xilinx, .qsf for Intel) once the target FPGA board is confirmed, enabling immediate synthesis and deployment.

## 13 REFERENCES


1. S. Palnitkar, *Verilog HDL: A Guide to Digital Design and Synthesis*, 2nd ed. Prentice Hall PTR, 2003.

2. M. D. Ciletti, *Advanced Digital Design with the Verilog HDL*, 2nd ed. Prentice Hall, 2010.

3. Terasic Technologies, "DE10-Lite User Manual," Terasic Inc., 2017. [Online]. Available: `https://www.terasic.com.tw/cgi-bin/page/archive.pl?Language=English&CategoryNo=167&No=1046`

4. Xilinx Inc., "7 Series FPGAs Configurable Logic Block User Guide (UG474)," Xilinx, 2016.

5. J. Wakerly, *Digital Design: Principles and Practices*, 5th ed. Pearson, 2017.

6. Icarus Verilog Project, "Icarus Verilog Documentation." [Online]. Available: `https://steveicarus.github.io/iverilog/`